01 — Limpieza de datos

Give Me Some Credit — Análisis de riesgo crediticio

Autor: Mario Zarza Zárraga
Objetivo de este notebook: preparar el dataset de entrenamiento (`cs-training.csv`) para el EDA y el modelado. Cada decisión está documentada con el *por qué*, no solo el *qué*.

Dataset: competencia Give Me Some Credit (Kaggle).
Unidad de observación: un cliente.
Variable objetivo: `SeriousDlqin2yrs` — 1 si el cliente cayó en mora grave (≥90 días) en los siguientes 2 años.

0. Librerías y carga

In [1]:
```
import pandas as pd
import numpy as np
from pathlib import Path

# Rutas relativas al root del proyecto
ROOT = Path("..")
DATA = ROOT / "data"
RAW = DATA / "cs-training.csv"
CLEAN = DATA / "cs-training-limpio.csv"

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
```

1. Carga y renombre de columnas

Las columnas originales vienen en inglés y con nombres largos. Las renombramos a **snake_case en español** para que el resto del proyecto sea más legible. La primera columna es un índice sin nombre; la descartamos.

In [2]:
```
# Cargamos el CSV crudo
df = pd.read_csv(RAW)

# La primera columna es un índice (Unnamed: 0); no aporta información
df = df.drop(columns=["Unnamed: 0"])

print(f"Forma inicial: {df.shape[0]:,} filas × {df.shape[1]} columnas")
df.head()
```

```
Forma inicial: 150,000 filas × 11 columnas
```

Out[2]:

	SeriousDlqin2yrs	RevolvingUtilizationOfUnsecuredLines	age	NumberOfTime3 59DaysPastDueNotWor
0	1	0.7661	45	
1	0	0.9572	40	
2	0	0.6582	38	
3	0	0.2338	30	
4	0	0.9072	49	

Tabla de mapeo de columnas

Original (inglés)	Nuevo nombre (español)	Significado
`SeriousDlqin2yrs`	`mora_grave`	1 = cayó en mora ≥90 días en los siguientes 2 años
`RevolvingUtilizationOfUnsecuredLines`	`uso_credito_revolvente`	% de uso de líneas revolving (tarjetas, etc.
`age`	`edad`	Edad del cliente en años
`NumberOfTime30-59DaysPastDueNotWorse`	`atrasos_30_59`	Veces con atraso de 30-59 días
`DebtRatio`	`ratio_deuda`	Pagos mensuales / ingreso (o deuda absoluta si no hay ingreso)
`MonthlyIncome`	`ingreso_mensual`	Ingreso mensual reportado
`NumberOfOpenCreditLinesAndLoans`	`lineas_credito_abiertas`	Número de líneas de crédito y

Original (inglés)	Nuevo nombre (español)	Significado
		préstamos abiertos
`NumberOfTimes90DaysLate`	`atrasos_90`	Veces con atraso de 90+ días
`NumberRealEstateLoansOrLines`	`prestamos_hipotecarios`	Préstamos o líneas hipotecarias
`NumberOfTime60-89DaysPastDueNotWorse`	`atrasos_60_89`	Veces con atraso de 60-89 días
`NumberOfDependents`	`dependientes`	Número de dependientes

In [3]:
```
# Diccionario de renombre
rename_map = {
    "SeriousDlqin2yrs": "mora_grave",
    "RevolvingUtilizationOfUnsecuredLines": "uso_credito_revolvente",
    "age": "edad",
    "NumberOfTime30-59DaysPastDueNotWorse": "atrasos_30_59",
    "DebtRatio": "ratio_deuda",
    "MonthlyIncome": "ingreso_mensual",
    "NumberOfOpenCreditLinesAndLoans": "lineas_credito_abiertas",
    "NumberOfTimes90DaysLate": "atrasos_90",
    "NumberRealEstateLoansOrLines": "prestamos_hipotecarios",
    "NumberOfTime60-89DaysPastDueNotWorse": "atrasos_60_89",
    "NumberOfDependents": "dependientes",
}

df = df.rename(columns=rename_map)
print("Columnas:", list(df.columns))
```

```
Columnas: ['mora_grave', 'uso_credito_revolvente', 'edad', 'atrasos_30_59',
'ratio_deuda', 'ingreso_mensual', 'lineas_credito_abiertas', 'atrasos_90',
'prestamos_hipotecarios', 'atrasos_60_89', 'dependientes']
```

In [4]:
```
# Tipos de dato
print("=== dtypes ===")
print(df.dtypes)
print("\n=== nulos por columna ===")
nulos = df.isnull().sum().to_frame("nulos")
nulos["pct"] = (nulos["nulos"] / len(df) * 100).round(2)
print(nulos)
```

```
=== dtypes ===
mora_grave                  int64
uso_credito_revolvente    float64
edad                        int64
atrasos_30_59               int64
ratio_deuda               float64
ingreso_mensual           float64
lineas_credito_abiertas     int64
atrasos_90                  int64
prestamos_hipotecarios      int64
atrasos_60_89               int64
dependientes              float64
dtype: object

=== nulos por columna ===
                          nulos     pct
mora_grave                    0  0.0000
uso_credito_revolvente        0  0.0000
edad                          0  0.0000
atrasos_30_59                 0  0.0000
ratio_deuda                   0  0.0000
ingreso_mensual           29731 19.8200
lineas_credito_abiertas       0  0.0000
atrasos_90                    0  0.0000
prestamos_hipotecarios        0  0.0000
atrasos_60_89                 0  0.0000
dependientes               3924  2.6200
```

In [5]:
```
# Resumen estadístico
df.describe().T
```

Out[5]:

	count	mean	std	min	25%	
mora_grave	150,000.0000	0.0668	0.2497	0.0000	0.0000	0
uso_credito_revolvente	150,000.0000	6.0484	249.7554	0.0000	0.0299	0
edad	150,000.0000	52.2952	14.7719	0.0000	41.0000	52
atrasos_30_59	150,000.0000	0.4210	4.1928	0.0000	0.0000	0
ratio_deuda	150,000.0000	353.0051	2,037.8185	0.0000	0.1751	0
ingreso_mensual	120,269.0000	6,670.2212	14,384.6742	0.0000	3,400.0000	5,400
lineas_credito_abiertas	150,000.0000	8.4528	5.1460	0.0000	5.0000	8
atrasos_90	150,000.0000	0.2660	4.1693	0.0000	0.0000	0
prestamos_hipotecarios	150,000.0000	1.0182	1.1298	0.0000	0.0000	1
atrasos_60_89	150,000.0000	0.2404	4.1552	0.0000	0.0000	0
dependientes	146,076.0000	0.7572	1.1151	0.0000	0.0000	0

Lectura rápida del `describe`

- **Tasa de mora global:** ~6.7% (`mora_grave` mean ≈ 0.067). El dataset está **desbalanceado**.
- **Ingreso:** hay nulos (~20%) y un máximo de ~3 millones → outliers fuertes.
- **Atrasos:** máximos de 98 → valores sospechosos (los revisamos abajo).
- **Uso revolvente:** máximo 50,708 con media esperada entre 0 y 1 → outliers.
- **Edad:** mínimo 0 → error de captura.

2. Edad = 0 → borrar la fila

Una sola fila tiene `edad = 0` . Es un **error de captura**, no un bebé con tarjeta de crédito. La borramos.

In [6]:
```
# Contamos y eliminamos la fila con edad 0
n_edad_0 = (df["edad"] == 0).sum()
print(f"Filas con edad = 0: {n_edad_0}")

df = df[df["edad"] != 0].copy()
print(f"Filas eliminadas: {n_edad_0}")
print(f"Forma después de borrar edad 0: {df.shape[0]:,} filas")
```

```
Filas con edad = 0: 1
Filas eliminadas: 1
Forma después de borrar edad 0: 149,999 filas
```

3. Códigos 96 / 98 en atrasos → bandera + imputación

En las tres columnas de atraso (`atrasos_30_59` , `atrasos_60_89` , `atrasos_90`) aparecen los valores **96** y **98**. No son conteos reales: saltan de ~13/17 directo a 96/98, y los mismos clientes tienen el código en las **tres** columnas a la vez. Son **códigos de sistema** (dato no disponible / caso especial).

Importante: estos clientes tienen una tasa de mora ~8× mayor que el resto. **No los borramos**: el código es una señal de riesgo. Creamos una bandera y luego imputamos el conteo con la mediana.

In [7]:
```
cols_atraso = ["atrasos_30_59", "atrasos_60_89", "atrasos_90"]

# Valores únicos altos de cada columna (para ver el salto a 96/98)
for c in cols_atraso:
    unicos = sorted(df[c].unique())
    print(f"{c}: últimos valores = {unicos[-10:]}")
```

```
atrasos_30_59: últimos valores = [np.int64(6), np.int64(7), np.int64(8), np.
int64(9), np.int64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(9
6), np.int64(98)]
atrasos_60_89: últimos valores = [np.int64(3), np.int64(4), np.int64(5), np.
int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(11), np.int64(96),
np.int64(98)]
atrasos_90: últimos valores = [np.int64(9), np.int64(10), np.int64(11), np.i
nt64(12), np.int64(13), np.int64(14), np.int64(15), np.int64(17), np.int64(9
6), np.int64(98)]
```

In [8]:
```
# Clientes con 96 o 98 en las TRES columnas de atraso
mask_codigo = (
    df["atrasos_30_59"].isin([96, 98])
    & df["atrasos_60_89"].isin([96, 98])
    & df["atrasos_90"].isin([96, 98])
)

n_codigo = int(mask_codigo.sum())
print(f"Clientes con código 96/98 en las 3 columnas: {n_codigo}")

# Comparación de tasa de mora
tasa_codigo = df.loc[mask_codigo, "mora_grave"].mean()
tasa_resto = df.loc[~mask_codigo, "mora_grave"].mean()
print(f"Tasa de mora (código especial): {tasa_codigo:.2%}")
print(f"Tasa de mora (resto):           {tasa_resto:.2%}")
print(f"Ratio: {tasa_codigo / tasa_resto:.1f}× más mora en el grupo con códi
```

```
Clientes con código 96/98 en las 3 columnas: 269
Tasa de mora (código especial): 54.65%
Tasa de mora (resto):           6.60%
Ratio: 8.3× más mora en el grupo con código
```

In [9]:
```
# Bandera: el código en sí aporta información al modelo
df["codigo_atraso_especial"] = mask_codigo.astype(int)

# Reemplazamos 96/98 por NaN y luego imputamos con la mediana de cada column
for c in cols_atraso:
    df.loc[df[c].isin([96, 98]), c] = np.nan
    mediana = df[c].median()
    n_imp = int(df[c].isna().sum())
    df[c] = df[c].fillna(mediana)
    print(f"{c}: imputados {n_imp} valores con mediana = {mediana}")

print(f"\nBandera codigo_atraso_especial = 1: {int(df['codigo_atraso_especia
```

```
atrasos_30_59: imputados 269 valores con mediana = 0.0
atrasos_60_89: imputados 269 valores con mediana = 0.0
atrasos_90: imputados 269 valores con mediana = 0.0

Bandera codigo_atraso_especial = 1: 269 clientes
```

4. Ingreso mensual faltante → bandera + mediana

Hay ~29,731 nulos (~20%). **Antes de imputar**, comprobamos si el faltante es aleatorio (MCAR) o no.

In [10]:
```
# Grupo con / sin ingreso reportado (antes de imputar)
miss = df["ingreso_mensual"].isna()

comparacion = pd.DataFrame({
    "con_ingreso": [
        int((~miss).sum()),
        df.loc[~miss, "mora_grave"].mean(),
        df.loc[~miss, "edad"].median(),
        df.loc[~miss, "ratio_deuda"].median(),
    ],
    "sin_ingreso": [
        int(miss.sum()),
        df.loc[miss, "mora_grave"].mean(),
        df.loc[miss, "edad"].median(),
        df.loc[miss, "ratio_deuda"].median(),
    ],
}, index=["n_clientes", "tasa_mora", "edad_mediana", "ratio_deuda_mediana"])

print("¿El faltante es aleatorio? Comparación de grupos:")
comparacion
```

```
¿El faltante es aleatorio? Comparación de grupos:
```

Out[10]:

	con_ingreso	sin_ingreso
n_clientes	120,268.0000	29,731.0000
tasa_mora	0.0695	0.0561
edad_mediana	51.0000	57.0000
ratio_deuda_mediana	0.2960	1,159.0000

Hallazgo: el faltante **no** es aleatorio

Indicador	Con ingreso	Sin ingreso
Tasa de mora	~6.9%	~5.6%
Edad mediana	51	57
`ratio_deuda` mediana	~0.30	~1,159

Los clientes sin ingreso son mayores, caen un poco menos en mora, y su `ratio_deuda` es absurdo como proporción. Lo más probable: **sin ingreso, el sistema dejó la deuda en pesos absolutos** en vez de dividir. El faltante "contamina" también otra columna.

Decisión:

1. Bandera `ingreso_faltante` (1/0) — el faltante aporta señal.
2. Imputar `ingreso_mensual` con la **mediana** (5,400), no con la media (~6,670), porque outliers de hasta ~3 M inflan la media.
3. Marcar `ratio_deuda_dudoso` cuando el ingreso faltaba, y **capar** `ratio_deuda` en su percentil 99 para que los valores extremos no dominen gráficas ni modelos lineales.

In [11]:
```
# Estadísticos del ingreso (solo valores observados)
ingreso_obs = df["ingreso_mensual"].dropna()
print(f"Nulos de ingreso: {df['ingreso_mensual'].isna().sum():,} ({df['ingre
print(f"Mediana ingreso:  {ingreso_obs.median():,.0f}")
print(f"Media ingreso:    {ingreso_obs.mean():,.2f}")
print(f"Máximo ingreso:   {ingreso_obs.max():,.0f}")
```

```
Nulos de ingreso: 29,731 (19.8%)
Mediana ingreso:  5,400
Media ingreso:    6,670.23
Máximo ingreso:   3,008,750
```

In [12]:
```
# Bandera + imputación con mediana
mediana_ingreso = df["ingreso_mensual"].median()

df["ingreso_faltante"] = df["ingreso_mensual"].isna().astype(int)
# Cuando faltaba el ingreso, el ratio_deuda es dudoso (posible deuda absolut
df["ratio_deuda_dudoso"] = df["ingreso_faltante"]  # misma definición; nombr

df["ingreso_mensual"] = df["ingreso_mensual"].fillna(mediana_ingreso)

print(f"Imputados con mediana = {mediana_ingreso:,.0f}")
print(f"Bandera ingreso_faltante = 1: {int(df['ingreso_faltante'].sum()):,}
```

```
Imputados con mediana = 5,400
Bandera ingreso_faltante = 1: 29,731 clientes
```

In [13]:
```
# Cap de ratio_deuda en el percentil 99
p99_ratio = df["ratio_deuda"].quantile(0.99)
n_cap_ratio = int((df["ratio_deuda"] > p99_ratio).sum())

print(f"Percentil 99 de ratio_deuda: {p99_ratio:,.2f}")
print(f"Filas que se capan: {n_cap_ratio:,} ({n_cap_ratio / len(df):.2%})")
print(f"Máximo antes del cap: {df['ratio_deuda'].max():,.2f}")

df["ratio_deuda"] = df["ratio_deuda"].clip(upper=p99_ratio)
print(f"Máximo después del cap: {df['ratio_deuda'].max():,.2f}")
```

```
Percentil 99 de ratio_deuda: 4,979.08
Filas que se capan: 1,500 (1.00%)
Máximo antes del cap: 329,664.00
Máximo después del cap: 4,979.08
```

> **Caveat para el EDA/modelo:** cuando `ratio_deuda_dudoso = 1`, el valor de `ratio_deuda` (aunque capado) no es comparable con el del

> resto. En el modelado conviene incluir la bandera; en el EDA, segmentar o excluir ese grupo al interpretar el ratio.

5. Dependientes faltantes → imputar 0

Hay 3,924 nulos. La mediana de `dependientes` es **0**. Imputar con 0 es la opción más simple y alineada con la distribución (muchos clientes no reportan / no tienen dependientes).

In [14]:
```
n_dep_nulos = int(df["dependientes"].isna().sum())
mediana_dep = df["dependientes"].median()
print(f"Nulos en dependientes: {n_dep_nulos:,}")
print(f"Mediana de dependientes: {mediana_dep}")

# Imputamos con 0 (coincide con la mediana)
df["dependientes"] = df["dependientes"].fillna(0)
print(f"Nulos restantes en dependientes: {int(df['dependientes'].isna().sum(
```

```
Nulos en dependientes: 3,924
Mediana de dependientes: 0.0
Nulos restantes en dependientes: 0
```

6. Uso de crédito revolvente > 1 → bandera + cap

Un uso revolvente "normal" está entre 0 y 1 (0%–100%). Hay valores enormes (máximo 50,708). Creamos una bandera `uso_revolvente_gt1` y capamos en el percentil 99 para no distorsionar el EDA ni modelos sensibles a escala.

In [15]:
```
uso = df["uso_credito_revolvente"]
print(f"Máximo uso revolvente: {uso.max():,.2f}")
print(f"Valores > 1: {int((uso > 1).sum()):,} ({(uso > 1).mean():.2%})")
print(f"Percentil 99: {uso.quantile(0.99):,.4f}")
```

```
Máximo uso revolvente: 50,708.00
Valores > 1: 3,321 (2.21%)
Percentil 99: 1.0930
```

In [16]:
```
# Bandera de valores anómalos (> 1) y cap en p99
df["uso_revolvente_gt1"] = (df["uso_credito_revolvente"] > 1).astype(int)

p99_uso = df["uso_credito_revolvente"].quantile(0.99)
n_cap_uso = int((df["uso_credito_revolvente"] > p99_uso).sum())

df["uso_credito_revolvente"] = df["uso_credito_revolvente"].clip(upper=p99_u

print(f"Bandera uso_revolvente_gt1 = 1: {int(df['uso_revolvente_gt1'].sum())
print(f"Cap en p99 = {p99_uso:,.4f} ({n_cap_uso:,} filas afectadas)")
print(f"Máximo después del cap: {df['uso_credito_revolvente'].max():,.4f}")
```

```
Bandera uso_revolvente_gt1 = 1: 3,321
Cap en p99 = 1.0930 (1,500 filas afectadas)
Máximo después del cap: 1.0930
```

7. Chequeos finales y guardado

In [17]:
```
print("=== Nulos restantes ===")
print(df.isnull().sum())
print(f"\nForma final: {df.shape[0]:,} filas × {df.shape[1]} columnas")
print("Columnas nuevas (banderas): codigo_atraso_especial, ingreso_faltante,
print(f"\nTasa de mora final: {df['mora_grave'].mean():.2%}")
df.head()
```

```
=== Nulos restantes ===
mora_grave                  0
uso_credito_revolvente      0
edad                        0
atrasos_30_59               0
ratio_deuda                 0
ingreso_mensual             0
lineas_credito_abiertas     0
atrasos_90                  0
prestamos_hipotecarios      0
atrasos_60_89               0
dependientes                0
codigo_atraso_especial      0
ingreso_faltante            0
ratio_deuda_dudoso          0
uso_revolvente_gt1          0
dtype: int64

Forma final: 149,999 filas × 15 columnas
Columnas nuevas (banderas): codigo_atraso_especial, ingreso_faltante, ratio_
deuda_dudoso, uso_revolvente_gt1

Tasa de mora final: 6.68%
```

Out[17]:

	mora_grave	uso_credito_revolvente	edad	atrasos_30_59	ratio_deuda	ingreso_men
0	1	0.7661	45	2.0000	0.8030	9,120.(
1	0	0.9572	40	0.0000	0.1219	2,600.(
2	0	0.6582	38	1.0000	0.0851	3,042.(
3	0	0.2338	30	0.0000	0.0360	3,300.(
4	0	0.9072	49	1.0000	0.0249	63,588.(

In [18]:
```
# Guardamos el dataset limpio
df.to_csv(CLEAN, index=False)
print(f"Guardado en: {CLEAN.resolve()}")
print(f"Tamaño: {CLEAN.stat().st_size / 1e6:.2f} MB")
```

```
Guardado en: /workspace/gmsc/data/cs-training-limpio.csv
Tamaño: 9.35 MB
```

Resumen de decisiones

Problema	Decisión	Por qué
Columna índice `Unnamed: 0`	Eliminar	No aporta información
`edad = 0` (1 fila)	Borrar la fila	Error de captura; una sola observación
Atrasos = 96 / 98 (269 clientes)	Bandera `codigo_atraso_especial` + reemplazar por NaN e imputar mediana	Son códigos de sistema, no conteos; el grupo tiene ~54.6% de mora vs ~6.6% del resto
`ingreso_mensual` nulo (~20%)	Bandera `ingreso_faltante` + imputar mediana (5,400)	El faltante no es aleatorio (edad, mora y ratio distintos); la media se infla por outliers (~3 M)
`ratio_deuda` en filas sin ingreso	Bandera `ratio_deuda_dudoso` + cap en p99	Sin ingreso, el ratio parece deuda absoluta (mediana ~1,159 vs ~0.30)
`dependientes` nulo (3,924)	Imputar 0	Coincide con la mediana; opción simple y alineada a la distribución
`uso_credito_revolvente` extremos (máx. 50,708)	Bandera `uso_revolvente_gt1` + cap en p99	Valores > 1 son sospechosos; el cap evita que distorsionen EDA/modelos

Siguiente notebook: `02_eda.ipynb` — exploración de la tasa de mora por edad, ingreso y endeudamiento.